

Try Mermaid Advanced Editor — OSS users get 10% off with code JS26

Get started

Class diagrams

"In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among objects."

-Wikipedia

The class diagram is the main building block of object-oriented modeling. It is used for general conceptual modeling of the structure of the application, and for detailed modeling to translate the models into programming code. Class diagrams can also be used for data modeling. The classes in a class diagram represent both the main elements, interactions in the application, and the classes to be programmed.

Mermaid can render class diagrams.

Code:

mermaid

```
----
title: Animal example
----
classDiagram
    note "From Duck till Zebra"
    Animal <|-- Duck
    note for Duck "can fly<br>can swim<br>can dive<br>can help in debuggi
    Animal <|-- Fish
    Animal <|-- Zebra
    Animal : +int age
    Animal : +String gender
    Animal: +isMammal()
    Animal: +mate()
    class Duck{
        +String beakColor
```

```
+swim()  
+quack()  
}  
class Fish{  
  -int sizeInFeet  
  -canEat()  
}  
class Zebra{  
  +bool is_wild  
  +run()  
}
```

⌘ + Enter | [Run ▶](#)

Syntax

Class

UML provides mechanisms to represent class members, such as attributes and methods, and additional information about them. A single instance of a class in the diagram contains three compartments:

- The top compartment contains the name of the class. It is printed in bold and centered, and the first letter is capitalized. It may also contain optional annotation text describing the nature of the class.
- The middle compartment contains the attributes of the class. They are left-aligned and the first letter is lowercase.
- The bottom compartment contains the operations the class can execute. They are also left-aligned and the first letter is lowercase.

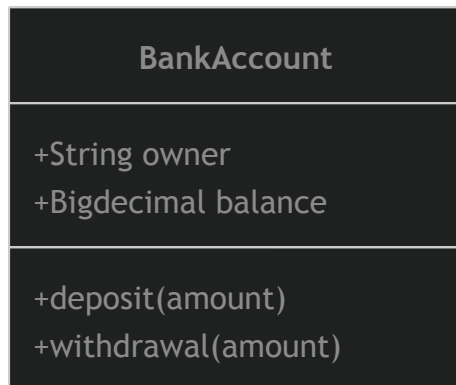
Code:

```
mermaid  
---  
title: Bank example  
---  
classDiagram  
    class BankAccount  
    BankAccount : +String owner  
    BankAccount : +BigDecimal balance  
    BankAccount : +deposit(amount)
```

```
BankAccount : +withdrawal(amount)
```

⌘ + Enter | [Run ▶](#)

Bank example



Define a class

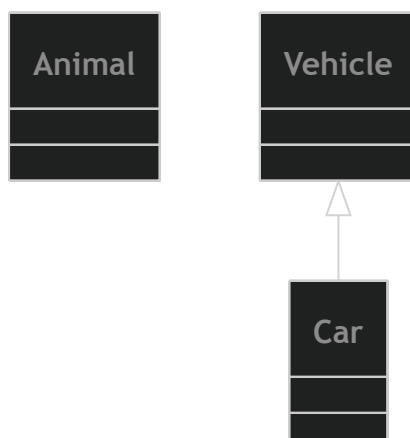
There are two ways to define a class:

- Explicitly using keyword **class** like `class Animal` which would define the `Animal` class.
- Via a **relationship** which defines two classes at a time along with their relationship. For instance, `Vehicle <|-- Car`.

Code:

```
classDiagram
    class Animal
    Vehicle <|-- Car
```

mermaid

⌘ + Enter | [Run ▶](#)

Naming convention: a class name should be composed only of alphanumeric characters (including unicode), underscores, and dashes (-).

Class labels

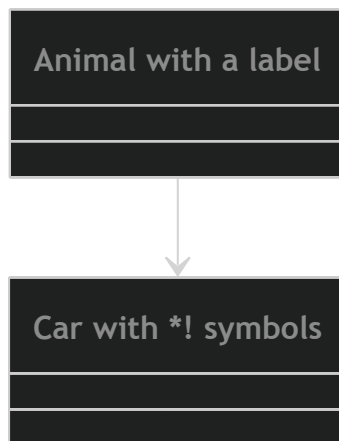
In case you need to provide a label for a class, you can use the following syntax:

Code:

```
classDiagram
    class Animal["Animal with a label"]
    class Car["Car with *! symbols"]
    Animal --> Car
```

mermaid

⌘ + Enter | [Run ▶](#)



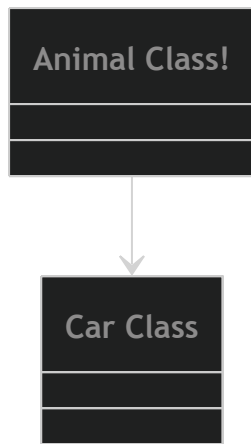
You can also use backticks to escape special characters in the label:

Code:

```
classDiagram
    class `Animal Class!`
    class `Car Class`
    `Animal Class!` --> `Car Class`
```

mermaid

⌘ + Enter | [Run ▶](#)



Defining Members of a class

UML provides mechanisms to represent class members such as attributes and methods, as well as additional information about them.

Mermaid distinguishes between attributes and functions/methods based on if the **parenthesis** `()` are present or not. The ones with `()` are treated as functions/methods, and all others as attributes.

There are two ways to define the members of a class, and regardless of whichever syntax is used to define the members, the output will still be same. The two different ways are :

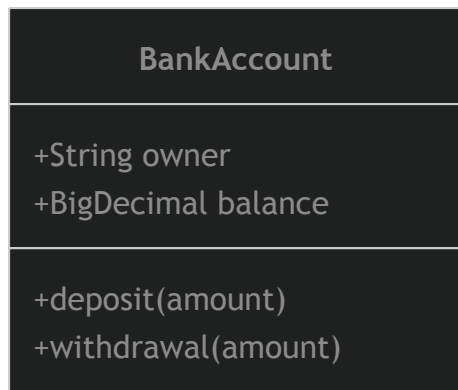
- Associate a member of a class using `:` (colon) followed by member name, useful to define one member at a time. For example:

Code:

```
classDiagram
class BankAccount
BankAccount : +String owner
BankAccount : +BigDecimal balance
BankAccount : +deposit(amount)
BankAccount : +withdrawal(amount)
```

mermaid

⌘ + Enter | **Run** ▶

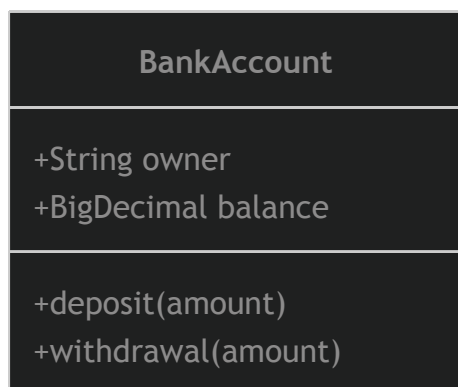


- Associate members of a class using `{ }` brackets, where members are grouped within curly brackets. Suitable for defining multiple members at once. For example:

Code:

```
classDiagram
class BankAccount{
    +String owner
    +BigDecimal balance
    +deposit(amount)
    +withdrawal(amount)
}
```

mermaid

⌘ + Enter | [Run ▶](#)

Return Type

Optionally you can end a method/function definition with the data type that will be returned (note: there must be a space between the final `)` and the return type). An example:

Code:

```
classDiagram
class BankAccount{
```

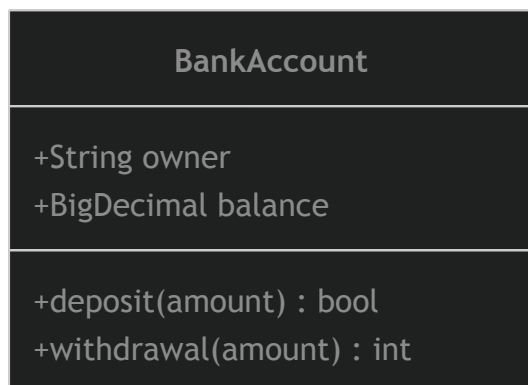
mermaid

```

+String owner
+BigDecimal balance
+deposit(amount) bool
+withdrawal(amount) int
}

```

⌘ + Enter | [Run ▶](#)



Generic Types

Generics can be represented as part of a class definition, and for class members/return types. In order to denote an item as generic, you enclose that type within `~` (tilde). **Nested** type declarations such as `List<List<int>>` are supported, though generics that include a comma are currently not supported. (such as `List<List<K, V>>`)

note when a generic is used within a class definition, the generic type is NOT considered part of the class name. i.e.: for any syntax which required you to reference the class name, you need to drop the type part of the definition. This also means that mermaid does not currently support having two classes with the same name, but different generic types.

Code:

```

classDiagram
class Square~Shape~{
    int id
    List~int~ position
    setPoints(List~int~ points)
    getPoints() List~int~
}

```

mermaid

```

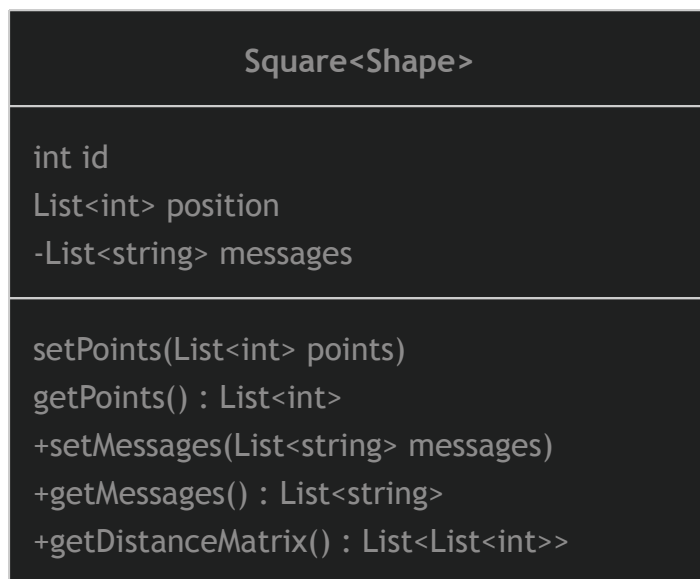
Square : -List~string~ messages
Square : +setMessages(List~string~ messages)

```

```

Square : +getMessages() List~string~
Square : +getDistanceMatrix() List~List~int~~

```

⌘ + Enter | [Run ▶](#)

Visibility

To describe the visibility (or encapsulation) of an attribute or method/function that is a part of a class (i.e. a class member), optional notation may be placed before that members' name:

- **+** Public
- **-** Private
- **#** Protected
- **~** Package/Internal

note you can also include additional *classifiers* to a method definition by adding the following notation to the *end* of the method, i.e.: after the **()** or after the return type:

- ***** Abstract e.g.: `someAbstractMethod()*` or `someAbstractMethod() int*`
- **\$** Static e.g.: `someStaticMethod()$` or `someStaticMethod() String$`

note you can also include additional *classifiers* to a field definition by adding the following notation to the very end:

- **\$** Static e.g.: `String someField$`

Defining Relationship

A relationship is a general term covering the specific types of logical connections found on class and object diagrams.

```
[classA] [Arrow] [ClassB]
```

There are eight different types of relations defined for classes under UML which are currently supported:

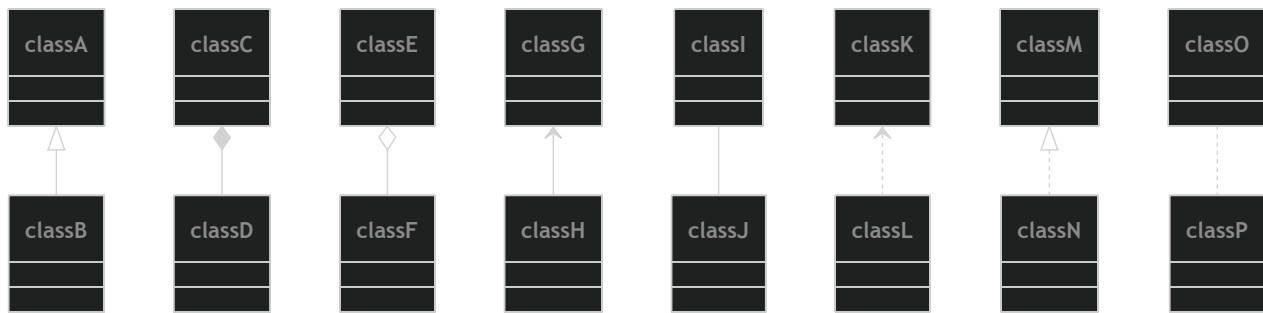
Type	Description
< --	Inheritance
*--	Composition
o--	Aggregation
-->	Association
--	Link (Solid)
..>	Dependency
.. >	Realization
..	Link (Dashed)

Code:

mermaid

```
classDiagram
classA <|-- classB
classC *-- classD
classE o-- classF
classG <-- classH
classI -- classJ
classK <.. classL
classM <|.. classN
classO .. classP
```

⌘ + Enter | Run ▶



We can use the labels to describe the nature of the relation between two classes.

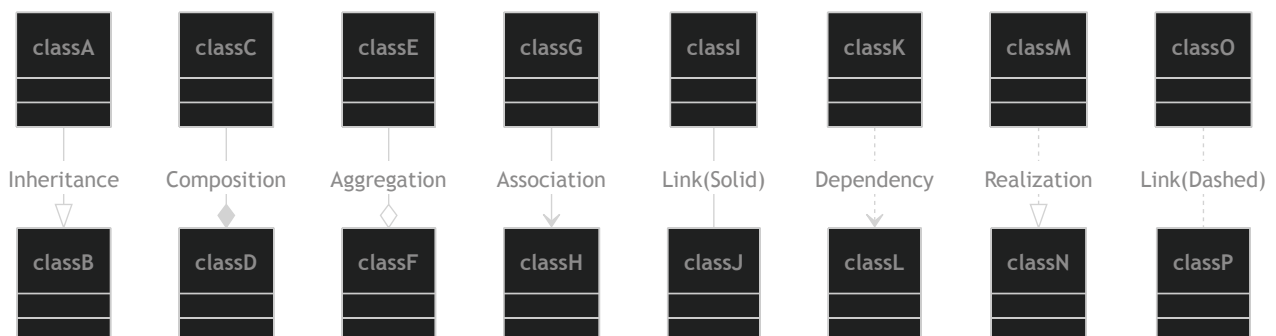
Also, arrowheads can be used in the opposite direction as well:

Code:

```
classDiagram
classA --|> classB : Inheritance
classC --* classD : Composition
classE --o classF : Aggregation
classG --> classH : Association
classI -- classJ : Link(Solid)
classK ..> classL : Dependency
classM ..|> classN : Realization
classO .. classP : Link(Dashed)
```

mermaid

⌘ + Enter | [Run ▶](#)



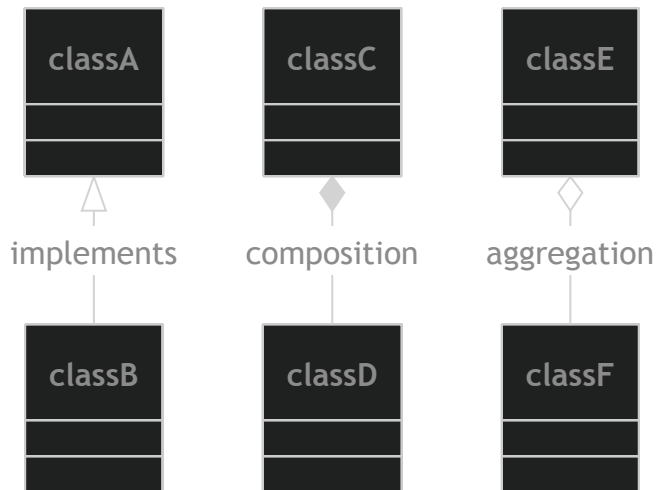
Labels on Relations

It is possible to add label text to a relation:

```
[classA] [Arrow] [ClassB] : LabelText
```

Code:

```
classDiagram
classA <|-- classB : implements
classC *-- classD : composition
classE o-- classF : aggregation
```

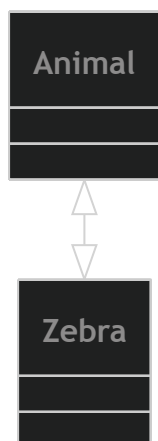
⌘ + Enter | [Run ▶](#)

Two-way relations

Relations can logically represent an N:M association:

Code:

```
classDiagram
    Animal <|--> Zebra
```

⌘ + Enter | [Run ▶](#)

Here is the syntax:

```
[Relation Type] [Link] [Relation Type]
```

Where **Relation Type** can be one of:

Type	Description
<	Inheritance
*	Composition
o	Aggregation
>	Association
<	Association
>	Realization

And **Link** can be one of:

Type	Description
--	Solid
..	Dashed

Lollipop Interfaces

Classes can also be given a special relation type that defines a lollipop interface on the class. A lollipop interface is defined using the following syntax:

- **bar ()-- foo**
- **foo --() bar**

The interface (bar) with the lollipop connects to the class (foo).

Note: Each interface that is defined is unique and is meant to not be shared between classes / have multiple edges connecting to it.

Code:

```
classDiagram
    bar ()-- foo
```

mermaid

⌘ + Enter | **Run** ▶



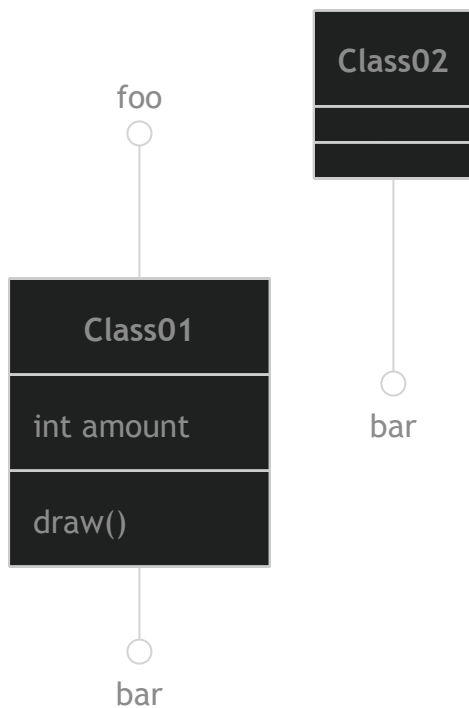
Code:

mermaid

```
classDiagram
    class Class01 {
        int amount
        draw()
    }
    Class01 --() bar
    Class02 --() bar

    foo ()-- Class01
```

⌘ + Enter | [Run ▶](#)



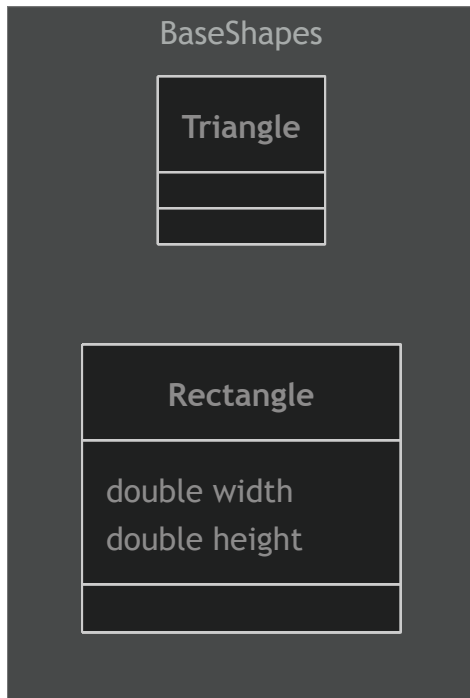
Define Namespace

A namespace groups classes.

Code:

```
classDiagram
namespace BaseShapes {
  class Triangle
  class Rectangle {
    double width
    double height
  }
}
```

⌘ + Enter | [Run ▶](#)



Cardinality / Multiplicity on relations

Multiplicity or cardinality in class diagrams indicates the number of instances of one class that can be linked to an instance of the other class. For example, each company will have one or more employees (not zero), and each employee currently works for zero or one companies.

Multiplicity notations are placed near the end of an association.

The different cardinality options are :

- **1** Only 1
- **0..1** Zero or One
- **1..*** One or more

- * Many
- n n (where n>1)
- 0..n zero to n (where n>1)
- 1..n one to n (where n>1)

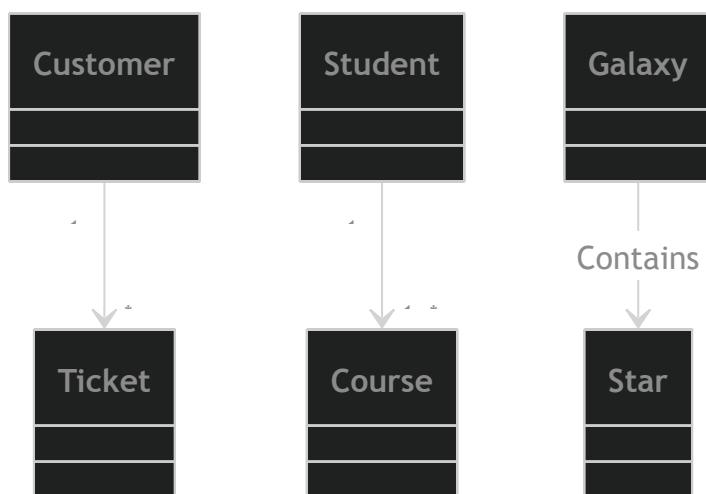
Cardinality can be easily defined by placing the text option within quotes " " before or after a given arrow. For example:

```
[classA] "cardinality1" [Arrow] "cardinality2" [ClassB]:LabelText
```

Code:

```
classDiagram
    Customer "1" --> "*" Ticket
    Student "1" --> "1..*" Course
    Galaxy --> "many" Star : Contains
```

mermaid

⌘ + Enter | [Run ▶](#)

Annotations on classes

It is possible to annotate classes with markers to provide additional metadata about the class. This can give a clearer indication about its nature. Some common annotations include:

- <<Interface>> To represent an Interface class
- <<Abstract>> To represent an abstract class

- `<<Service>>` To represent a service class
- `<<Enumeration>>` To represent an enum

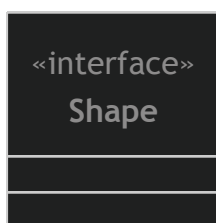
Annotations are defined within the opening `<<` and closing `>>` . There are three ways to add an annotation to a class, and in all cases the output will be the same:

- **Inline** with the class definition:

Code:

```
classDiagram
  class Shape <<interface>>
```

mermaid

⌘ + Enter | [Run ▶](#)

- In a **separate line** after a class is defined:

Code:

```
classDiagram
  class Shape
  <<interface>> Shape
  Shape : noOfVertices
  Shape : draw()
```

mermaid

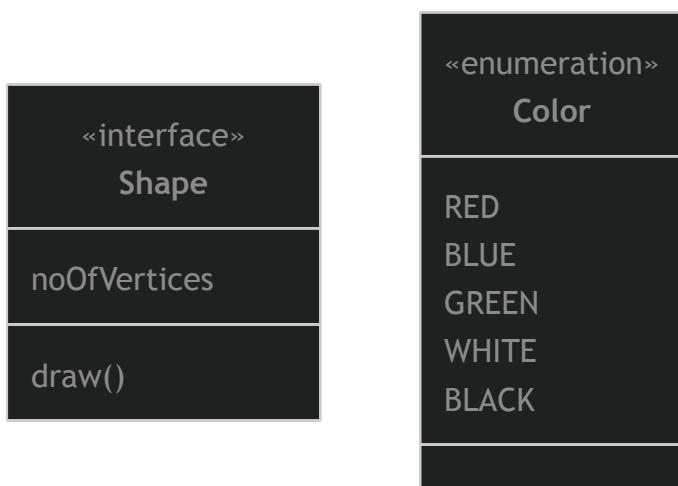
⌘ + Enter | [Run ▶](#)

- In a **nested structure** along with the class definition:

Code:

```
classDiagram
class Shape{
    <<interface>>
    noOfVertices
    draw()
}
class Color{
    <<enumeration>>
    RED
    BLUE
    GREEN
    WHITE
    BLACK
}
```

⌘ + Enter | [Run ▶](#)



Comments

Comments can be entered within a class diagram, which will be ignored by the parser. Comments need to be on their own line, and must be prefaced with `%%` (double percent signs). Any text until the next newline will be treated as a comment, including any class diagram syntax.

Code:

```
classDiagram
%% This whole line is a comment classDiagram class Shape <<interface>>
```

```
class Shape{
  <<interface>>
  noOfVertices
  draw()
}
```

⌘ + Enter | [Run ▶](#)

Setting the direction of the diagram

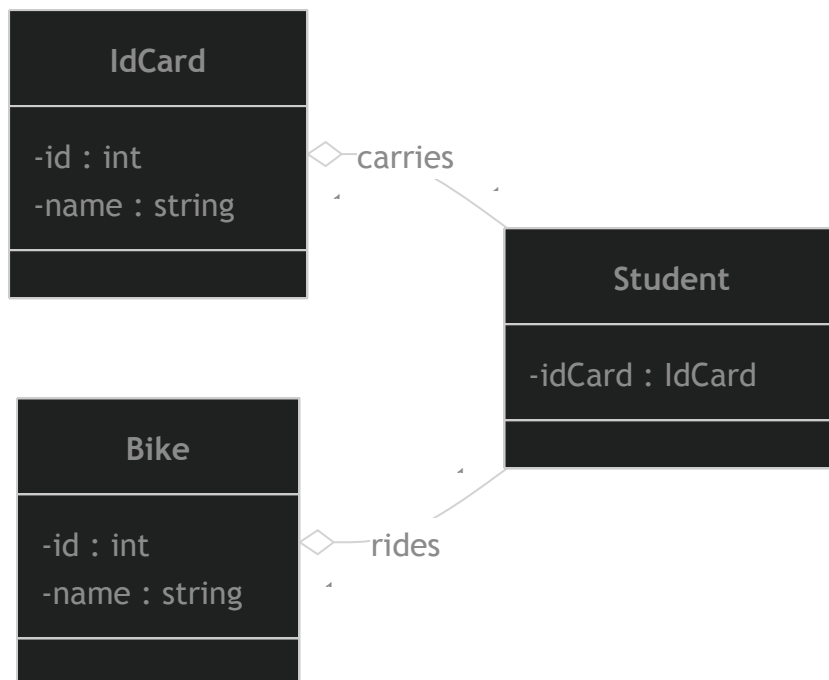
With class diagrams you can use the direction statement to set the direction in which the diagram will render:

Code:

mermaid

```
classDiagram
  direction RL
  class Student {
    -idCard : IdCard
  }
  class IdCard{
    -id : int
    -name : string
  }
  class Bike{
    -id : int
    -name : string
  }
  Student "1" --o "1" IdCard : carries
  Student "1" --o "1" Bike : rides
```

⌘ + Enter | [Run ▶](#)



Interaction

It is possible to bind a click event to a node. The click can lead to either a javascript callback or to a link which will be opened in a new browser tab. **Note:** This functionality is disabled when using `securityLevel='strict'` and enabled when using `securityLevel='loose'`.

You would define these actions on a separate line after all classes have been declared.

```

action className "reference" "tooltip"
click className call callback() "tooltip"
click className href "url" "tooltip"
  
```

- *action* is either `link` or `callback`, depending on which type of interaction you want to have called
- *className* is the id of the node that the action will be associated with
- *reference* is either the url link, or the function name for callback.
- (*optional*) *tooltip* is a string to be displayed when hovering over element (note: The styles of the tooltip are set by the class `.mermaidTooltip`.)
- note: callback function will be called with the `nodeId` as parameter.

Notes

It is possible to add notes on the diagram using `note "line1\nline2"`. A note can be added for a specific class using `note for <CLASS NAME> "line1\nline2"`.

Examples

Code:

```
classDiagram
    note "This is a general note"
    note for MyClass "This is a note for a class"
    class MyClass{
    }
```

mermaid

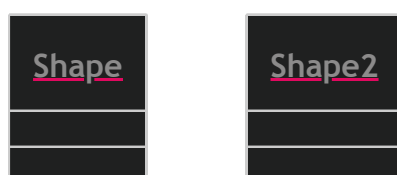
⌘ + Enter | [Run ▶](#)

URL Link:

Code:

```
classDiagram
    class Shape
    link Shape "https://www.github.com" "This is a tooltip for a link"
    class Shape2
    click Shape2 href "https://www.github.com" "This is a tooltip for a link"
```

mermaid

⌘ + Enter | [Run ▶](#)

Callback:

Code:

mermaid

```

classDiagram
class Shape
callback Shape "callbackFunction" "This is a tooltip for a callback"
class Shape2
click Shape2 call callbackFunction() "This is a tooltip for a callback"

```

⌘ + Enter | [Run ▶](#)

html

```

<script>
  const callbackFunction = function () {
    alert('A callback was triggered');
  };
</script>

```

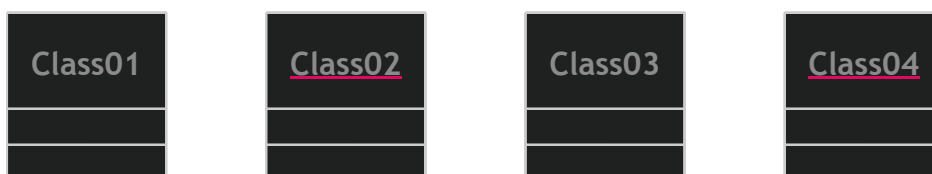
Code:

mermaid

```

classDiagram
  class Class01
  class Class02
  callback Class01 "callbackFunction" "Callback tooltip"
  link Class02 "https://www.github.com" "This is a link"
  class Class03
  class Class04
  click Class03 call callbackFunction() "Callback tooltip"
  click Class04 href "https://www.github.com" "This is a link"

```

⌘ + Enter | [Run ▶](#)

Success The tooltip functionality and the ability to link to urls are available from version 0.5.2.

Beginner's tip—a full example using interactive links in an HTML page:

```
<body>
  <pre class="mermaid">
    classDiagram
    Animal <|-- Duck
    Animal <|-- Fish
    Animal <|-- Zebra
    Animal : +int age
    Animal : +String gender
    Animal: +isMammal()
    Animal: +mate()
    class Duck{
      +String beakColor
      +swim()
      +quack()
    }
    class Fish{
      -int sizeInFeet
      -canEat()
    }
    class Zebra{
      +bool is_wild
      +run()
    }

    callback Duck "callback" "Tooltip"
    link Zebra "https://www.github.com" "This is a link"
  </pre>

  <script>
    const callback = function () {
      alert('A callback was triggered');
    };
    const config = {
      startOnLoad: true,
      securityLevel: 'loose',
    };
    mermaid.initialize(config);
  </script>
</body>
```

Styling

Styling a node

It is possible to apply specific styles such as a thicker border or a different background color to an individual node using the **style** keyword.

Note that notes and namespaces cannot be styled individually but do support themes.

Code:

```
classDiagram
    class Animal
    class Mineral
    style Animal fill:#f9f,stroke:#333,stroke-width:4px
    style Mineral fill:#bbf,stroke:#f66,stroke-width:2px,color:#fff,stroke-
```

mermaid

⌘ + Enter | Run ▶



Classes

More convenient than defining the style every time is to define a class of styles and attach this class to the nodes that should have a different look.

A class definition looks like the example below:

```
classDef className fill:#f9f,stroke:#333,stroke-width:4px;
```

Also, it is possible to define style to multiple classes in one statement:

```
classDef firstClassName,secondClassName font-size:12pt;
```

Attachment of a class to a node is done as per below:

```
cssClass "nodeId1" className;
```

It is also possible to attach a class to a list of nodes in one statement:

```
cssClass "nodeId1,nodeId2" className;
```

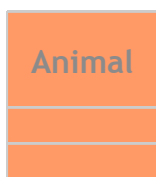
A shorter form of adding a class is to attach the classname to the node using the `:::` operator:

Code:

```
classDiagram
    class Animal:::someclass
    classDef someclass fill:#f96
```

mermaid

⌘ + Enter | [Run ▶](#)



Or:

Code:

```
classDiagram
    class Animal:::someclass {
        -int sizeInFeet
        -canEat()
    }
    classDef someclass fill:#f96
```

mermaid

⌘ + Enter | [Run ▶](#)



Default class

If a class is named default it will be applied to all nodes. Specific styles and classes should be defined afterwards to override the applied default styling.

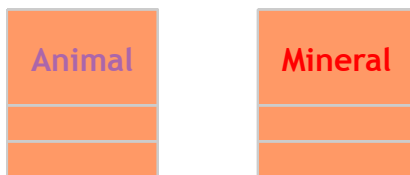
```
classDef default fill:#f9f,stroke:#333,stroke-width:4px;
```

Code:

mermaid

```
classDiagram
  class Animal:::pink
  class Mineral

classDef default fill:#f96,color:red
classDef pink color:#f9f
```

⌘ + Enter | [Run ▶](#)

CSS Classes

It is also possible to predefine classes in CSS styles that can be applied from the graph definition as in the example below:

Example style

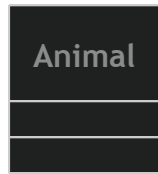
html

```
<style>
  .styleClass > * > g {
    fill: #ff0000;
    stroke: #ffff00;
    stroke-width: 4px;
  }
</style>
```

Example definition

Code:

```
classDiagram
    class Animal:::styleClass
```

⌘ + Enter | [Run ▶](#)

cssClasses cannot be added using this shorthand method at the same time as a relation statement.

Configuration

Members Box

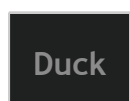
It is possible to hide the empty members box of a class node.

This is done by changing the **hideEmptyMembersBox** value of the class diagram configuration. For more information on how to edit the Mermaid configuration see the [configuration page](#).

Code:

```
---
config:
  class:
    hideEmptyMembersBox: true
---
classDiagram
    class Duck
```

mermaid

⌘ + Enter | [Run ▶](#)

[Edit this page on GitHub](#)

Previous page

[Sequence Diagram](#)

Next page

[State Diagram](#)

